

3. The Story Behind Clustered and Non-Clustered Indexes

"Indexes are not just performance optimizations — they directly influence how the optimizer thinks."

- Overview
- Basic Concepts
 - Page/row organization
 - Heap table
 - B-Tree structure
 - Seek vs scan
- Clustered Index
- Non-clustered Index
- Clustered vs Non-Clustered index comparison
- PostgreSQL Perspective
- Best practices when indexing
- References

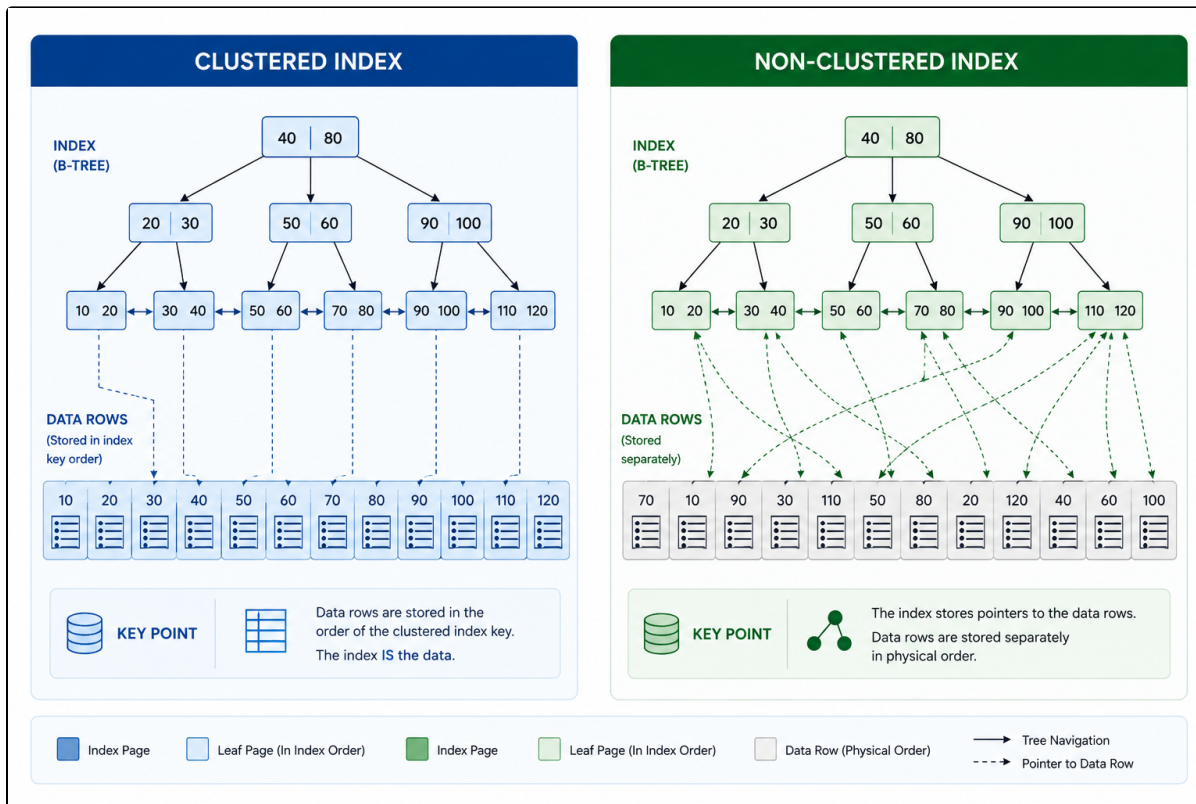
Overview

When working with large databases, ensuring optimal query performance is critical. Indexes are one of the most important database performance optimization mechanisms. They allow the database engine to locate data efficiently without scanning entire tables. Proper indexing can reduce query execution time from seconds to milliseconds, while poor indexing can lead to excessive I/O, CPU consumption, and degraded application performance. This session explores how clustered and non-clustered indexes work internally, how they affect query execution plans, and how they should be used in real-world systems.

Before discussing clustered and non-clustered indexes, it's important to understand that there are two different ways to classify indexes:

1. **Index structure (implementation)** – how the index is physically organized.
2. **Index organization (storage model)** – how the index relates to the table data.

Category	Examples
Index Structure	B-Tree, Hash, GIN, GiST, BRIN
Index Organization	Clustered, Non-Clustered



Basic Concepts

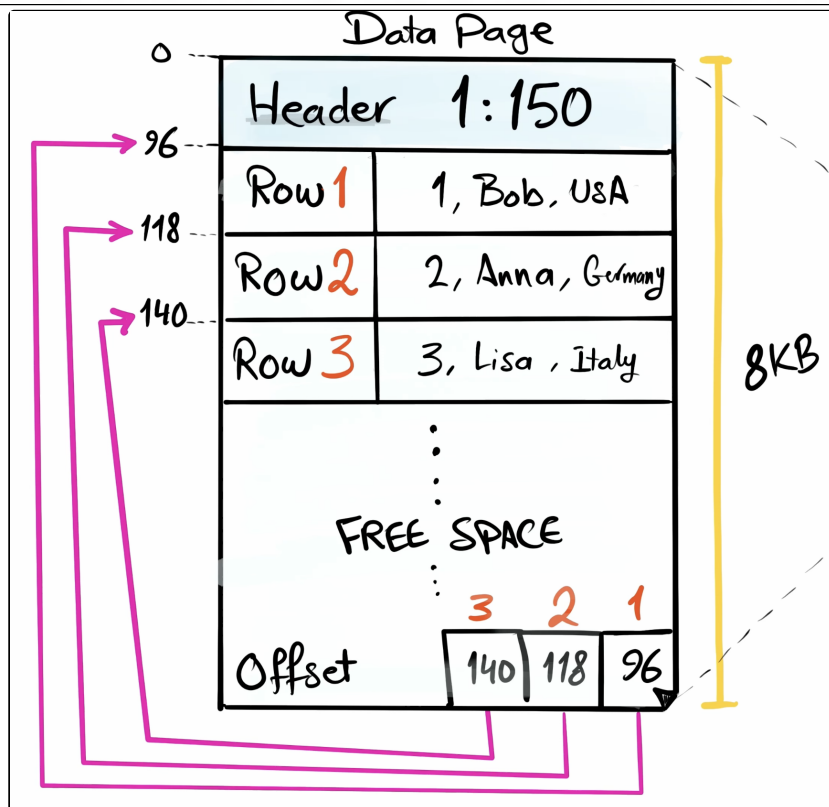
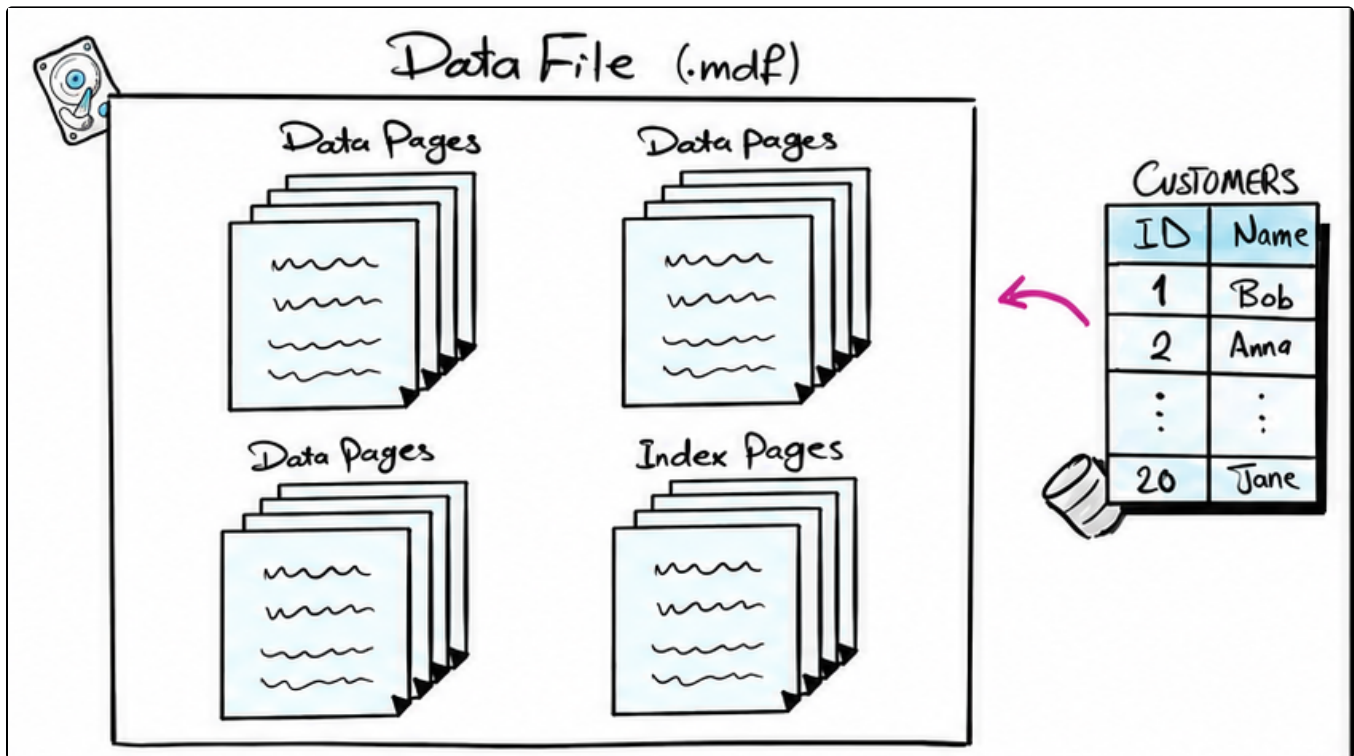
Before discussing index types, it is important to understand how data is stored and accessed inside a database.

Databases store data in pages and use specialized data structures to efficiently locate records. Understanding these fundamentals helps explain why some queries perform well while others require expensive scans.

Page/Row Organization

Smallest units of data storage in a database called Pages. Databases store data in fixed-size pages. For example, SQL Server uses 8 KB pages. It stores anything like data, indexes, metadata etc..

A table is physically stored as a collection of pages containing rows. When executing a query, the database reads pages from memory or disk.



Every row is identified by a physical location called a **Tuple Identifier (TID)**:

(page_number, row_offset)

Example:

(150, 140)

means:

- Page 150
- Row 3 within that page

Indexes help reduce the number of pages that must be accessed.

Heap Table

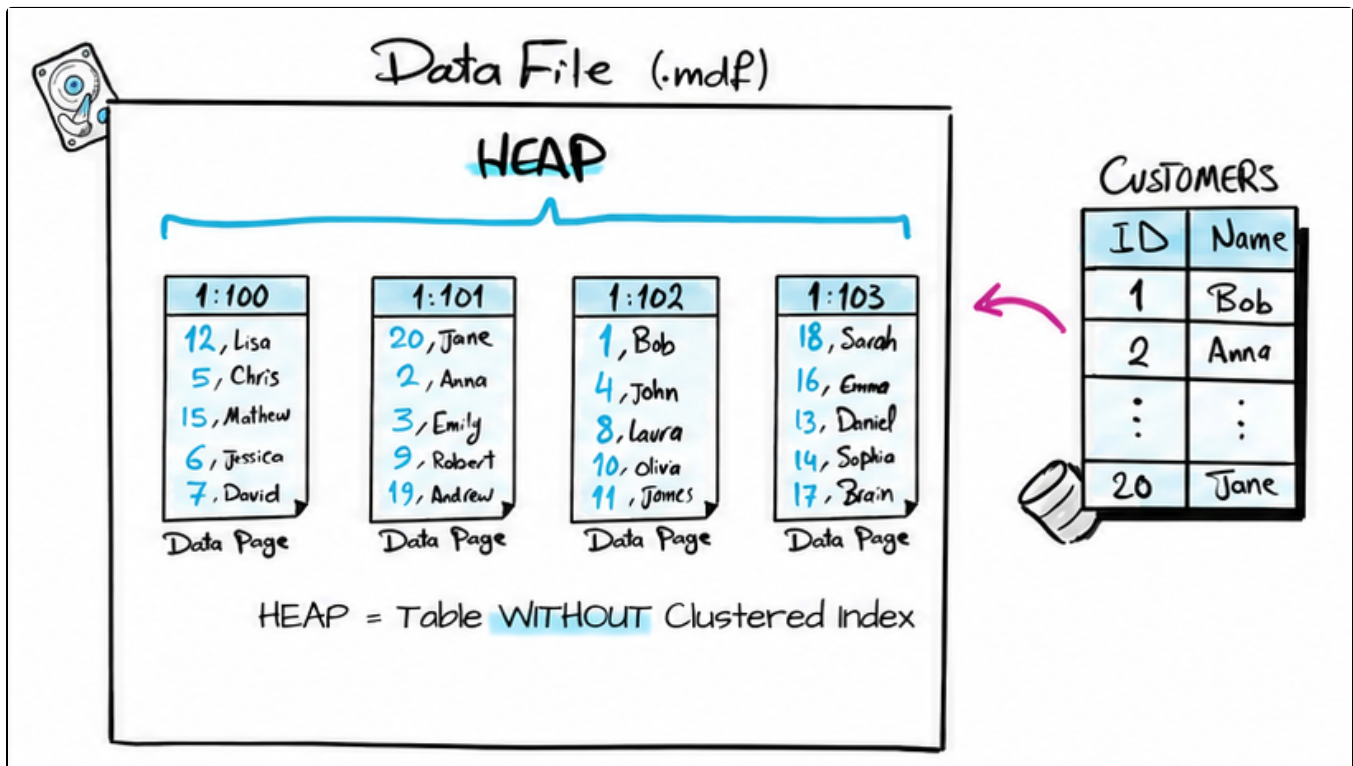
A heap is a table without a clustered index. Rows are stored wherever space is available, and there is no guaranteed physical ordering of data.

Characteristics:

- No logical row ordering
- Fast inserts
- Full table scans are often required
- Finding specific rows may require reading many pages

customers table without any index

```
CREATE TABLE customers (  
  id BIGINT,  
  name VARCHAR(100)  
);
```



Without a clustered index, this table is considered a heap.

B-Tree Structure

Most relational databases use B-Trees as the underlying structure for indexes.

A B-Tree consists of:

- Root page
- Intermediate pages
- Leaf pages

When searching for a value, the database starts at the root and navigates through the tree until the target value is found.

Benefits:

- Logarithmic search complexity
- Minimal disk reads
- Efficient range queries

The larger the table becomes, the more valuable the B-Tree structure becomes compared to scanning every row.

Seek vs Scan

A scan means the database reads a large portion of a table or index.

Examples:

- Table Scan
- Clustered Index Scan
- Index Scan

A seek means the database navigates directly to the required data using an index.

Examples:

- Clustered Index Seek
- Non-Clustered Index Seek

Generally:

Operation	Performance
Seek	Fast
Scan	Potentially expensive

However, scans are not always bad. For large result sets, a scan may be cheaper than many individual seeks.

Clustered Index

A clustered index determines the physical ordering of rows within a table.

Key characteristics:

- Only one clustered index per table
- Leaf pages contain actual table data
- Data is stored in index order
- Excellent for range queries

Example:

clustered index on orders table
<pre>CREATE CLUSTERED INDEX IX_Orders_OrderDate ON Orders(OrderDate);</pre>

Advantages:

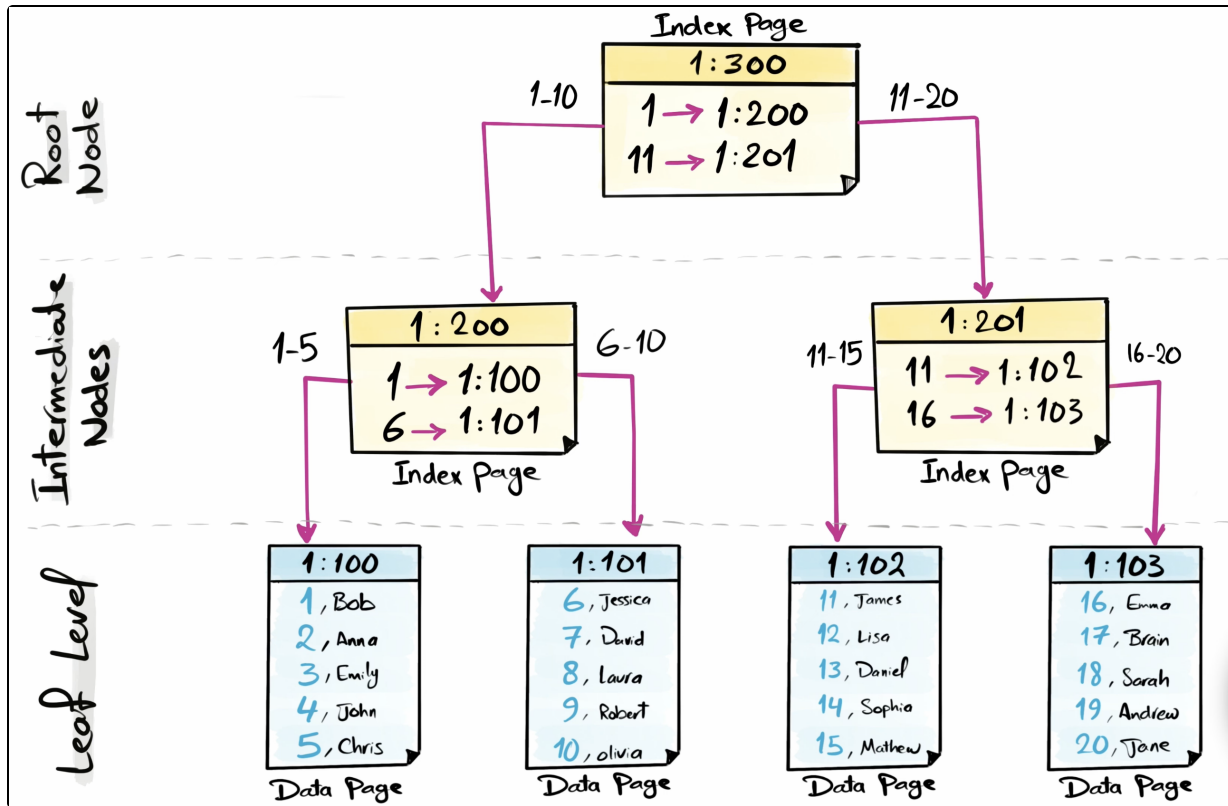
- Fast range searches
- Efficient sorting
- Reduced page reads

Disadvantages:

- Updates to key values can be expensive
- Random keys can cause fragmentation

Good clustered index candidates:

- Identity columns
- Sequential timestamps
- Frequently used range-filter columns



Non-clustered Index

A non-clustered index is a separate structure that contains indexed values and references to table rows.

Key characteristics:

- Multiple non-clustered indexes allowed
- Does not affect physical row order
- Optimized for selective lookups

Example:

non clustered index on customers table

```
CREATE INDEX IX_Customers_Email
ON Customers(Email);
```

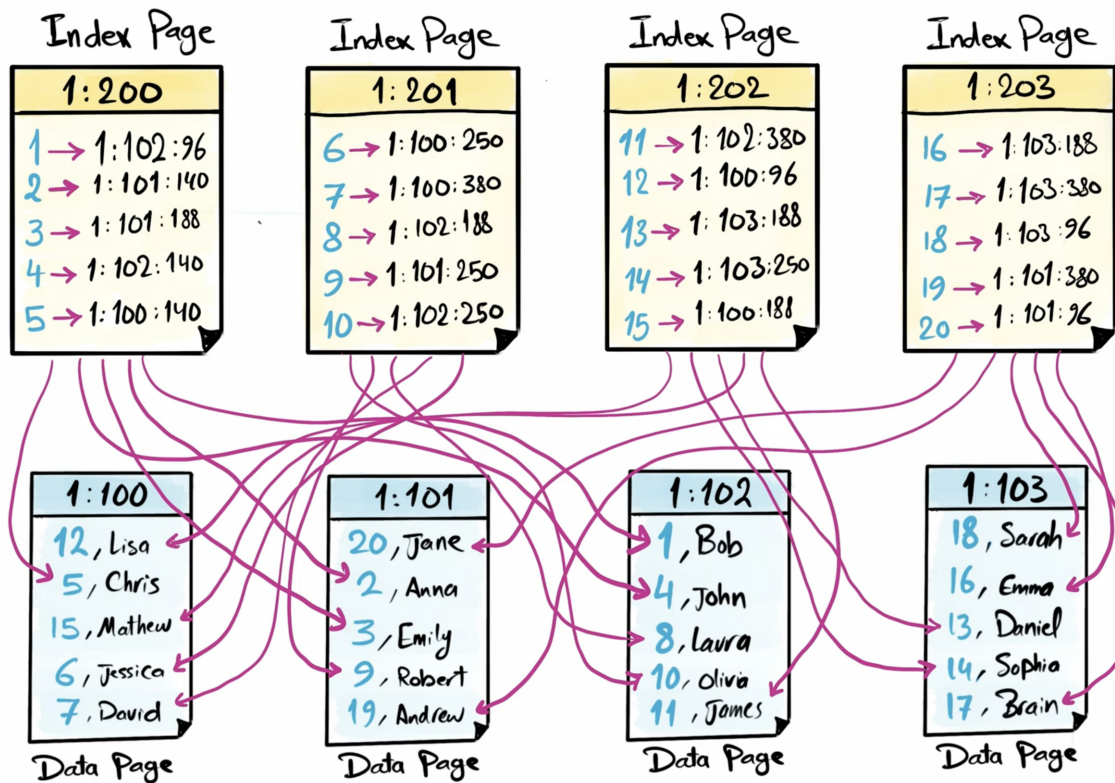
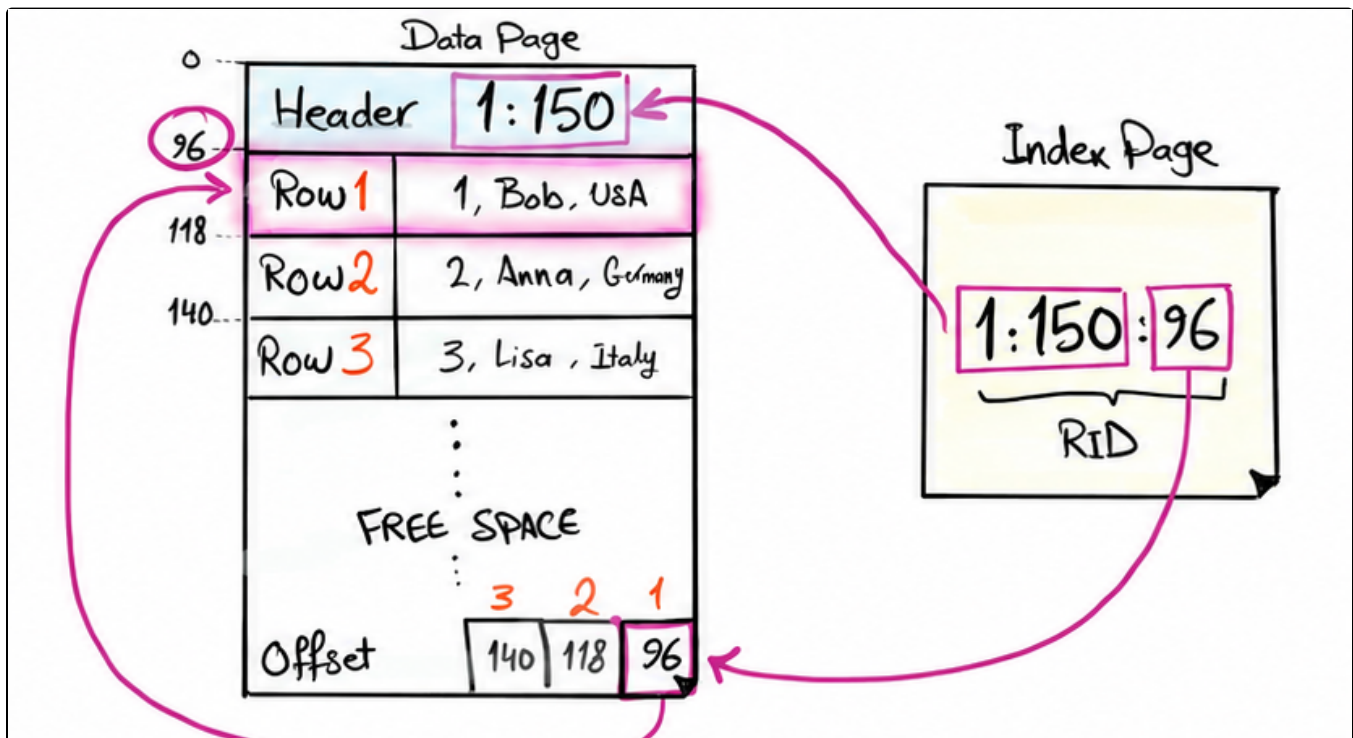
Advantages:

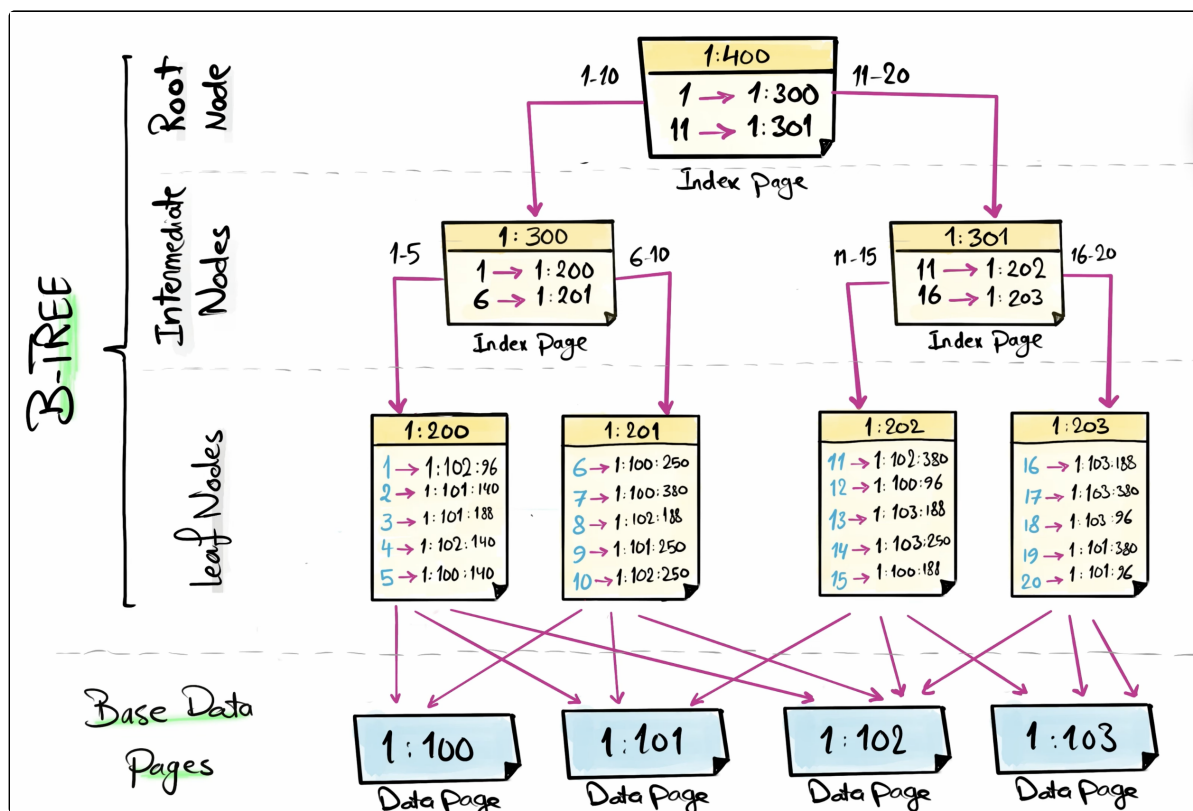
- Fast lookups
- Flexible indexing strategies
- Supports covering indexes

Disadvantages:

- Additional storage
- Increased write overhead

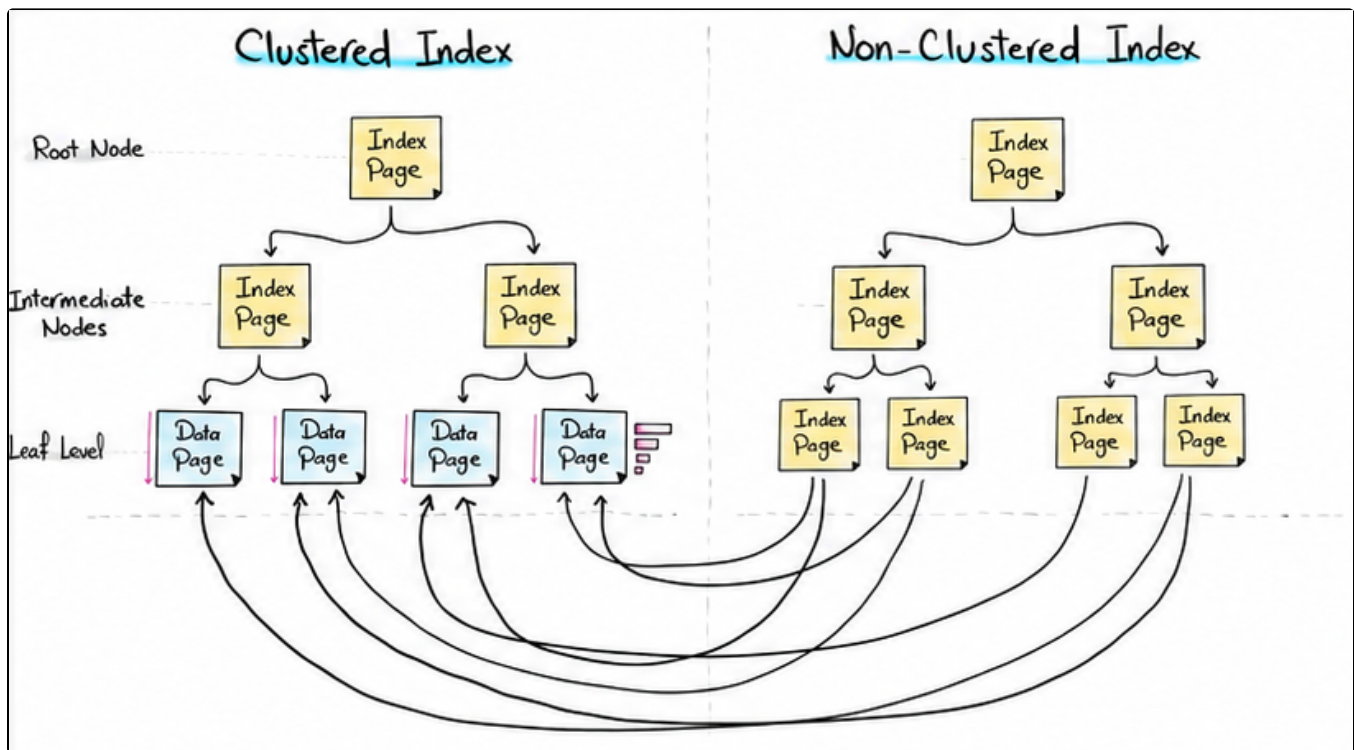
Every insert, update, or delete may require index maintenance.





Clustered vs Non-Clustered index comparison

Feature	Clustered	Non-Clustered
Physical row order	Yes	No
Count per table	One	Many
Leaf level contains	Actual data rows	Keys and pointers
Storage overhead	Lower	Additional storage
Range query performance	Excellent	Good
Lookup performance	Good	Excellent



Choosing the correct combination of clustered and non-clustered indexes is often more important than adding more indexes.

PostgreSQL Perspective

While SQL Server's clustered index determines the physical storage order of a table, PostgreSQL follows a fundamentally different architecture. PostgreSQL stores all tables as heaps and relies heavily on Multi-Version Concurrency Control (MVCC), visibility information, and advanced index access methods to achieve performance.

Understanding these differences is important when designing indexes and analyzing query plans.

MVCC and Visibility

One of PostgreSQL's most important concepts is MVCC. Each row contains metadata:

```
xmin  
xmax
```

These values determine:

- Which transaction created the row
- Which transaction deleted/updated the row

When a row is updated:

```
UPDATE customers  
SET name = 'John'  
WHERE id = 1;
```

PostgreSQL does not modify the row in place.

Instead:

```
Old Row -> Dead Version  
New Row -> Active Version
```

Benefits:

- Readers don't block writers
- Writers don't block readers

Trade-off:

- More storage consumption
- More maintenance work

VACUUM and Index Health

Because PostgreSQL continuously creates new row versions:

```
UPDATE  
DELETE
```

produce dead tuples.

Dead tuples remain until VACUUM removes them.

Without proper VACUUM:

- Table bloat increases
- Index bloat increases
- Query performance degrades
- Storage grows unnecessarily

A useful production mindset:

PostgreSQL performance depends not only on indexes but also on how effectively dead tuples are cleaned up.

HOT Updates (Heap-Only Tuple)

A very interesting PostgreSQL optimization.

When an UPDATE modifies only non-indexed columns:

```
UPDATE customers  
SET description = 'Updated'  
WHERE id = 1;
```

PostgreSQL may perform a HOT update.

Benefits:

- Indexes remain untouched
- Less WAL generation
- Faster updates
- Reduced index bloat

HOT updates are one reason PostgreSQL performance can remain stable under heavy write workloads.

Index Bloat

A common production problem.

Repeated updates can create:

Logical size: 10 GB

Physical size: 30 GB

Indexes become larger than necessary.

Symptoms:

- Increased disk reads
- Slower scans
- Poor cache utilization

Solutions:

- VACUUM
- REINDEX
- Better fillfactor settings

Fillfactor

PostgreSQL allows reserving free space on pages.

Example:

```
CREATE INDEX idx_customer
ON customers(email)
WITH (fillfactor = 80);
```

Meaning:

80% page usage

20% reserved space

Benefits:

- Fewer page splits
- Reduced fragmentation
- Better write performance

Useful for frequently updated tables.

PostgreSQL vs SQL Server: Key Architectural Difference

SQL Server	PostgreSQL
Clustered index stores table data	Tables always stored as heaps
One clustered index per table	No true clustered index
Key Lookup	Heap Fetch
Less dependence on VACUUM	VACUUM is critical
Physical ordering maintained	CLUSTER is one-time operation
Indexes point to rows through clustering key	Indexes point to TIDs

Best practices when indexing

Follow these practices for the best results:

Best Practices When Indexing

1. Choose the Right Clustered Index Key

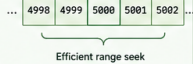
Clustered index keys should be **stable** and preferably **incrementally ordered** because updates can be expensive due to page splitting.



Best for range-based queries

Example:

```
SELECT * FROM Orders  
WHERE OrderID > 5000;
```



2. Use Nonclustered Indexes Wisely

Target frequent and selective queries that mainly use **WHERE**, **JOIN**, or **ORDER BY**.



- Create indexes on columns used together (composite indexes).
- Use **INCLUDE** to cover queries and avoid key lookups.

Example:

```
CREATE INDEX IX_Orders_CustomerDate  
ON Orders (CustomerID, OrderDate)  
INCLUDE (TotalAmount, Status);
```

Key Columns
CustomerID, OrderDate
INCLUDE Columns
TotalAmount, Status

3. Avoid Fragmentation

Fragmentation can cause severe performance problems.

Rebuild or reorganize indexes as needed to defragment page storage.



After **REORGANIZE** or **REBUILD**, update statistics to ensure performance improvement.



4. Avoid Over-Indexing

More indexes do not always mean better performance. Indexes improve **read speed** but can drastically affect **write speed**.

Read (SELECT)



Write (INSERT/UPDATE/DELETE)



Balance read and write costs based on your workload.

5. Index What You Query

Focus on columns that frequently appear in **WHERE**, **JOIN**, or **ORDER BY**.



Targeted Indexes



Don't index rarely used columns.



6. Match Indexing Strategy to Workload

Write-Heavy (OLTP)



- Many INSERT/UPDATE/DELETE
- Fewer indexes are better
- Focus on essential queries only

Goal: Fast Writes

Read-Heavy (OLAP)



- Many SELECT queries
- Multiple, well-designed indexes
- Cover common reporting and analytics queries

Goal: Fast Reads



Key Takeaway

Good indexing is about balance.

✓ Right columns

✓ Right indexes

✓ Right maintenance

✓ Right workload fit



Better performance,
lower cost, happier systems!

References

Books

1. Markus Winand, *SQL Performance Explained*
 - Excellent resource for understanding indexing, query optimization, and execution plans.
2. Kalen Delaney, *SQL Server Internals*
 - Detailed explanation of SQL Server storage engine internals, pages, B-Trees, clustered indexes, and query processing.
3. Grant Fritchey, *SQL Server Execution Plans*
 - Practical guide to reading and understanding execution plans.
4. Itzik Ben-Gan, *T-SQL Querying*
 - Covers indexing, query optimization, and SQL Server performance concepts.

Official Documentation

SQL Server

- Microsoft Documentation — Index Architecture and Design Guide
- Microsoft Documentation — Clustered and Nonclustered Indexes
- Microsoft Documentation — SQL Server Storage Engine Architecture

PostgreSQL

- PostgreSQL Documentation — Indexes
- PostgreSQL Documentation — MVCC (Multi-Version Concurrency Control)
- PostgreSQL Documentation — VACUUM
- PostgreSQL Documentation — CLUSTER

Videos

Video used in preparation:

- <https://youtu.be/BxAj3bl00-o>

Key Takeaway

Indexes are not just data structures that make queries faster. They directly influence how the optimizer estimates costs, chooses execution plans, and accesses data.